REFERENCE GUIDE & PRACTITIONER PLAYBOOK

# Applied Mathematics & Production LLM Cheat Sheet

**Author:** Akash Saini (Integrated M.Sc. Mathematics, BITS Pilani) | **Web:** https://akash12740.github.io | **Email:** aksainibits@gmail.com

## Module 1: Linear Algebra Made Visual (How AI Transforms Data)

In artificial intelligence, every word, image pixel, or user profile is stored as a list of numbers called a **vector**. Neural networks process these vectors by multiplying them with grids of numbers called **matrices**. Geometrically, multiplying a vector by a matrix simply stretches, rotates, shears, or flips space to reveal hidden patterns.

> **BEGINNER ANALOGY: THE RUBBER SHEET**
>
> Imagine drawing arrows on a transparent rubber sheet. A matrix transformation is the act of pulling the corners of that sheet evenly: straight grid lines stay straight and parallel, and the origin remains pinned in place, while every arrow shifts to a new length and direction.

- **2×2 Determinant ( $\mathbf{det(A) = ad - bc}$ ):** Measures how much a matrix scales area. If $\det(A) = 3$, a unit square stretches to an area of 3. If $\det(A) < 0$, the transformation flips space inside-out (like looking in a mirror). If $\det(A) = 0$, space is squashed flat onto a line or point, meaning information is permanently lost and the matrix cannot be inverted.
- **Matrix Trace ( $\mathbf{tr(A) = a + d}$ ):** The sum of the main diagonal entries. Remarkably, the trace always equals the sum of the matrix's eigenvalues ( $\lambda_1 + \lambda_2$ ), giving a quick sanity check on total directional expansion.
- **Eigenvectors & Eigenvalues ( $\mathbf{Av = \lambda v}$ ):** When a matrix transforms space, almost every arrow gets knocked off its original span. **Eigenvectors ( $\mathbf{v}$ )** are the special "axis" arrows that stay pointing along their exact original line—they only get scaled by a stretch factor called the **eigenvalue ( $\lambda$ )**.
- **Singular Value Decomposition ( $\mathbf{A = U\Sigma V^T}$ ) & LoRA Fine-Tuning:** SVD factors any rectangular weight matrix into two pure rotations ( $U, V^T$ ) and a diagonal stretch matrix ( $\Sigma$ ) ordered from strongest to weakest signal. In Large Language Models, massive weight updates are largely redundant. **LoRA (Low-Rank Adaptation)** freezes the giant base matrix $W$ and trains only two tiny rank-$r$ matrices ( $\Delta W = B \cdot A$ ), capturing the top singular directions while cutting trainable parameters by over 99%.

**WORKED NUMERICAL EXAMPLE: ANALYZING MATRIX A = [[3, 1], [0, 2]]**

Given $a = 3, b = 1, c = 0, d = 2$ :

- **Step 1 — Determinant:** $\det(A) = (3)(2) - (1)(0) = 6 - 0 = 6$ . Area scales by 6× with no orientation flip ( $6 > 0$ ).
- **Step 2 — Trace:** $\text{tr}(A) = 3 + 2 = 5$ .
- **Step 3 — Eigenvalues:** Solve $\det(A - \lambda I) = (3 - \lambda)(2 - \lambda) = 0$ , yielding $\lambda_1 = 3$ and $\lambda_2 = 2$ . Notice $\lambda_1 + \lambda_2 = 5 = \text{tr}(A)$ and $\lambda_1 \cdot \lambda_2 = 6 = \det(A)$ .
- **Step 4 — Eigenvectors:** For $\lambda_1 = 3$ , $(A - 3I)v = 0$ gives $v_1 = [1, 0]^T$ . For $\lambda_2 = 2$ , $(A - 2I)v = 0$ gives $v_2 = [-1, 1]^T$ .

## Module 2: Vector Search & Zero-Hallucination RAG Math

To let an LLM answer questions from private documents without guessing, Retrieval-Augmented Generation (RAG) converts every text paragraph into an **embedding vector**—an arrow in high-dimensional space (e.g., 768 or 1,536 dimensions) where semantically similar ideas point in nearly the same direction.

**BEGINNER ANALOGY: COMPASS BEARINGS IN A LIBRARY**

Imagine a giant library where every topic has a compass direction: "finance" points North, "biology" points East, and "biotech startups" points Northeast. Two sentences with the same meaning point along the exact same compass bearing, even if one sentence is a short tweet and the other is a long paragraph.

- **Cosine Similarity Formula:** Measures the angle $\theta$ between query vector $u$ and document vector $v$ , ignoring arrow length:
  $\cos(\theta) = (u \cdot v) / (\|u\| \|v\|) = (\sum u_i v_i) / (\sqrt{(\sum u_i^2)} \cdot \sqrt{(\sum v_i^2)})$ , ranging from $-1$ (opposite) to $+1$ (identical meaning).
- **Why L2-Normalization Accelerates FAISS:** Computing square roots and divisions across millions of vectors during live search is expensive. By pre-normalizing every stored vector to unit length ( $\|u\| = 1$ and $\|v\| = 1$ ) during indexing, the denominator becomes $1 \times 1 = 1$ . Vector databases like **FAISS** then compute exact cosine similarity using a single hardware-accelerated dot product: $\cos(\theta) = u \cdot v$ .

**WORKED NUMERICAL EXAMPLE: UNIT-VECTOR DOT PRODUCT**

Suppose a query embedding is $u = [0.6, 0.8]$ and a document chunk embedding is $v = [0.8, 0.6]$ :

- **Step 1 — Verify Unit Lengths (L2 Norms):** $\|u\| = \sqrt{(0.6^2 + 0.8^2)} = \sqrt{(0.36 + 0.64)} = 1.0$, and $\|v\| = \sqrt{(0.8^2 + 0.6^2)} = 1.0$.
- **Step 2 — Fast Dot Product:** $\cos(\theta) = u \cdot v = (0.6 \times 0.8) + (0.8 \times 0.6) = 0.48 + 0.48 = 0.96$.
- **Step 3 — Gate Decision:** Since $0.96 \geq 0.70$, the chunk passes the relevance threshold and is injected into the LLM context.

**PRODUCTION CHECKLIST: ZERO-HALLUCINATION RAG GUARDRAILS**

- **Similarity Threshold Gating ( $\cos(\theta) \geq 0.70$ ):** Never pass low-scoring noise chunks to the generator. If zero chunks exceed $0.70$, trigger a deterministic abstention (*"Insufficient evidence in retrieved records"*).
- **Mandatory Citation Grounding:** Require the LLM to append exact `[doc_id | chunk]` tags after every factual sentence, and programmatically verify that every cited ID existed in that request's retrieved context.

## Module 3: Optimization & Regularization (Learning Without Memorizing)

Training a machine learning model means adjusting millions of internal weights $w$ to minimize a **Loss Function** $L(w)$ that measures prediction error. Left unchecked, flexible models memorize training noise instead of learning general rules—a failure known as **overfitting**.

**BEGINNER ANALOGY: BLINDFOLDED MOUNTAIN DESCENT**

Imagine standing blindfolded on a foggy hillside trying to reach the lowest valley. You feel the slope under your boots (the **gradient** $\nabla L$) and take a step downhill. Your stride length is the **learning rate** ( $\alpha$ ): steps that are too tiny take forever; steps that are too giant leap right over the valley floor and bounce wildly up the opposite ridge.

- **Gradient Descent Update Rule:** $w_{new} = w_{old} - \alpha \nabla L(w_{old})$. Because the gradient $\nabla L$ points uphill toward steepest error increase, subtracting $\alpha \nabla L$ moves the weights steadily downhill toward minimum loss.

| Property | L1 Regularization (Lasso) | L2 Regularization (Ridge) |
|---|---|---|
| **Penalty Term** | $L_{total} = L_{data} + \lambda \sum \lvert w_i \rvert$ | $L_{total} = L_{data} + \lambda \sum w_i^2$ |
| **Geometric Effect** | Diamond constraint boundary; corners lie directly on coordinate axes. | Circular/spherical constraint boundary; shrinks weights smoothly toward zero. |
| **Weight Behavior** | **Induces Sparsity:** Forces weak feature weights to exact $0$ (built-in feature selection). | **Bounds Variance:** Keeps all weights small and balanced without zeroing them out. |

| Property | L1 Regularization (Lasso) | L2 Regularization (Ridge) |
|---|---|---|
| **Best Production Use** | Pruning noisy signals when only a few features truly drive the target. | High-dimensional **TF-IDF text features** where thousands of correlated words carry subtle signal. |

**WORKED NUMERICAL EXAMPLE: 1-STEP GRADIENT DESCENT**

Minimize the quadratic loss $L(w) = (w - 3)^2$ starting at $w_{old} = 0$ with learning rate $\alpha = 0.25$:

- **Step 1 — Compute Derivative (Gradient):** Using the chain rule, $\nabla L(w) = 2(w - 3)$.
- **Step 2 — Evaluate Slope at $w = 0$:** $\nabla L(0) = 2(0 - 3) = -6$ (negative slope means minimum lies to the right).
- **Step 3 — Update Weight:** $w_{new} = 0 - (0.25)(-6) = 0 + 1.5 = 1.5$. Loss drops immediately from $(0 - 3)^2 = 9$ down to $(1.5 - 3)^2 = 2.25$.

## Module 4: Production LLM Audits & Time-Series Forecasting Rules

Shipping reliable AI systems requires rigorous statistical discipline beyond model training—auditing step-by-step reasoning chains, preventing evaluation leakage, and enforcing stationarity on temporal business metrics.

- **Atomic Reasoning Trace Verification:** Evaluating only an LLM's final output misses dangerous silent failures—models frequently guess a plausible final answer while committing logical, arithmetic, or topological fallacies mid-chain. Production audits decompose every response into **atomic claims** and verify each step against retrieved evidence before trusting the conclusion.
- **Stratified K-Fold Cross-Validation (Preventing Data Leakage):** Randomly splitting imbalanced NLP datasets can starve validation folds of rare classes, while fitting TF-IDF vectorizers or scalers before splitting leaks test vocabulary into training. Fitting all preprocessing strictly inside each fold of **Stratified K-Fold CV** preserves exact class ratios across every split—enabling clean, reproducible **95% NLP classification accuracy** with zero leakage.
- **Time-Series Stationarity & Seasonal Differencing:** Classical forecasting models fail when a series exhibits trending mean or growing variance. Always run the **Augmented Dickey-Fuller (ADF) test** ($p < 0.05$ confirms stationarity). Applying first-order trend differencing ($d = 1$) plus 12-period seasonal differencing ($D = 1, s = 12$) in `SARIMA(1,1,1)(1,1,1,12)` stabilizes recurring annual cycles and cuts forecasting **RMSE by 60%** compared to un-differenced baselines.

**PRODUCTION CHECKLIST: PRE-RELEASE VALIDATION PROTOCOL**

- **Trace-Level LLM Eval:** Run LLM-as-a-Judge faithfulness scoring on decomposed reasoning steps, not just final strings.
- **Pipeline Encapsulation:** Wrap vectorizers, encoders, and classifiers inside a single `Pipeline` object before cross-validation.
- **Residual White-Noise Check:** Confirm SARIMA forecast residuals show zero remaining autocorrelation before deploying to production.


### Continue Learning & Building with Akash Saini

Explore interactive mathematics visualizations, open-source AI/ML projects, **1-on-1 tutoring (Mathematics, Science & English)**, and technical AI consulting at **https://akash12740.github.io** or book a 15-minute discovery session directly at **https://cal.com/akash-saini-12740/15min**.